Egg Segmentation and Tracking

CV Intern Task: Project Report (Tasks 1 to 4)

Prepared by: Sabaa Namazi

Links

Tracked video (public Google Drive link):
https://drive.google.com/file/d/1xBU2eQbkeQ98u3YnsAMKMuvQ8GlEF2BZ/view?usp=sharing

Source video (YouTube): https://youtu.be/Y4JyV94y2_A

Code: GitHub repository *egg-segmentation* (setup and run instructions in its README). This report is stored in the repository as report/report.pdf.

Summary

Task	Method	Main result	Page
1 Pretrained model	Grounding DINO (text prompt) finds the eggs, SAM turns each box into a mask	110 images, 138 eggs, an egg found in every image. The masks are model predictions and serve as ground truth in Tasks 2 and 3	2
2 Classical image processing	Threshold sweep over four channels, ellipse and edge scoring, brightness gate, GrabCut; scene-grouped 4:1 dev/eval split; 16 scored trials	Mean IoU 0.714 on dev (88 images) and 0.634 on eval (22 images, scored once)	9
3 Evaluation and analysis	Pixel and instance metrics, plots, overlays of the 12 worst eval images	Instance level: 21 TP, 7 FP, 6 FN (F1 0.76). Three images fail completely; causes: extra regions merged, the pale-egg assumption, shaded parts missed	21
4 Video tracking	The Task 2 pipeline on each frame plus a tracker that matches masks by distance and overlap	Only IDs 1 and 2 over 380 processed frames (10 s to 41.7 s); an ID swap was found and fixed. Identity under occlusion is inferred	28

Page numbers refer to this PDF. Each task section has its own footer numbering.

AI tools

I used Claude (Anthropic) as a coding and analysis assistant in all four tasks: to compare approaches, to draft code and diagnostics, and to help write this report. I ran every experiment myself, looked at the outputs and made the decisions. Each task section ends with its own disclosure.

Task 1: Egg Segmentation with Pretrained Models

1. Objective

Task 1 asks for segmentation masks of the eggs in every image of the dataset using a pretrained model. These masks are used later as pseudo ground truth in Task 3, so my main goals were clean boundaries, one mask per egg, and no background or hand pixels inside the masks.

2. Dataset

- 110 RGB images, all 640 × 640 px (Roboflow export, 77 train / 22 valid / 11 test folders).
- For Task 1 I ignored Roboflow's split and processed all 110 images together. The 4:1 dev/eval split is made in Task 2.
- Visual inspection showed a lot of variety: floor, tiles, a pan of water, a plastic plate, a hand, leaves and ash as backgrounds. Most images have one egg, but some have two or three, and some eggs are white on a near-white background.

3. Approach and why I chose it

I used a two-stage pipeline: **Grounding DINO** to find the eggs, then **SAM** (Segment Anything) to turn each detected box into a mask.

Why not SAM alone? SAM needs a prompt to know what to segment. Its automatic mode returns dozens of masks (fingers, palm, plates, background), and I would have needed hand-written rules to pick the egg masks. Those rules would be fragile on a dataset with this much background variety.

Why Grounding DINO? It is an open-vocabulary detector, so I can ask for "an egg" in plain text. This gives one box per egg without training anything or annotating images, and it handles images with several eggs in the same way as images with one.

Why box prompts for SAM? A box tightly specifies the object and gives cleaner masks than a single point, especially for touching eggs.

4. Technologies used

Tool	Purpose
Google Colab, T4 GPU (~15 GB)	Runs both transformer models quickly; a free GPU was enough for all 110 images
Python, PyTorch	Model inference
Hugging Face `transformers`	Loads Grounding DINO and SAM with one consistent API
`IDEA-Research/grounding-dino-base`	Text-prompted egg detection
`facebook/sam-vit-large`	Box-prompted mask generation
OpenCV, NumPy, Pillow	Mask post-processing, overlays, file I/O
Matplotlib	Inspection plots and report figures

I chose SAM ViT-Large as a balance: it is more accurate than the base model and still fits comfortably in T4 memory next to Grounding DINO.

5. Implementation

The script `task1_sam_segment.py` does the following for each image:

1. Runs Grounding DINO with the prompt `"an egg."` (box threshold 0.30, text threshold 0.25).
2. Drops any box that covers more than 85% of the image, which removes detections of the whole scene.
3. Passes the remaining boxes to SAM with `multimask_output=False` to get one mask per egg.
4. Merges the masks into a single instance map (uint16 PNG, 0 = background, 1..N = egg id). Where masks overlap, the egg with the lower id keeps the pixels.
5. Saves a red overlay image like the example in the task sheet, plus a JSON file with boxes and confidence scores.

I ran it with default parameters. I did not tune the thresholds, because the first run produced no systematic failures (see below).

6. Results

- Eggs were detected in **all 110 images** (0 images with no detection).
- Eggs per image: 93 images with 1 egg, 6 with 2 eggs, 11 with 3 eggs, for **138 eggs in total**.
- Detection confidence: mean 0.73, minimum 0.31.

How I checked quality. Because there is no human ground truth, I checked the output visually in three passes: (a) the first 12 overlays in alphabetical order, (b) the 12 images flagged by simple rules (several eggs, confidence below 0.4, or a mask covering under 3% or over 60% of the image), and (c) the smallest mask (image 22) next to its original.

What I saw:

- Mask boundaries follow the egg edges closely, and hands, plates, tiles and the pan are excluded.
- Multi-egg images are handled correctly, including three touching eggs (images 142, 144, 148) where each egg gets its own mask.
- Reflections and water in the steel pan were not detected as eggs (images 113, 114, 128, 129).
- Image 22 has the smallest mask (3% of the image). The original shows a real white egg on a white label, and the outline fits it well.

Figure 1 shows six representative cases from the 110 images: a single egg (100), three separate eggs (103), eggs in the steel pan (113), three touching eggs (144), an egg under leaves (10) and a white egg on a white label (22). The masks follow the egg boundary in all of them. Image 10 is the only imperfect one.

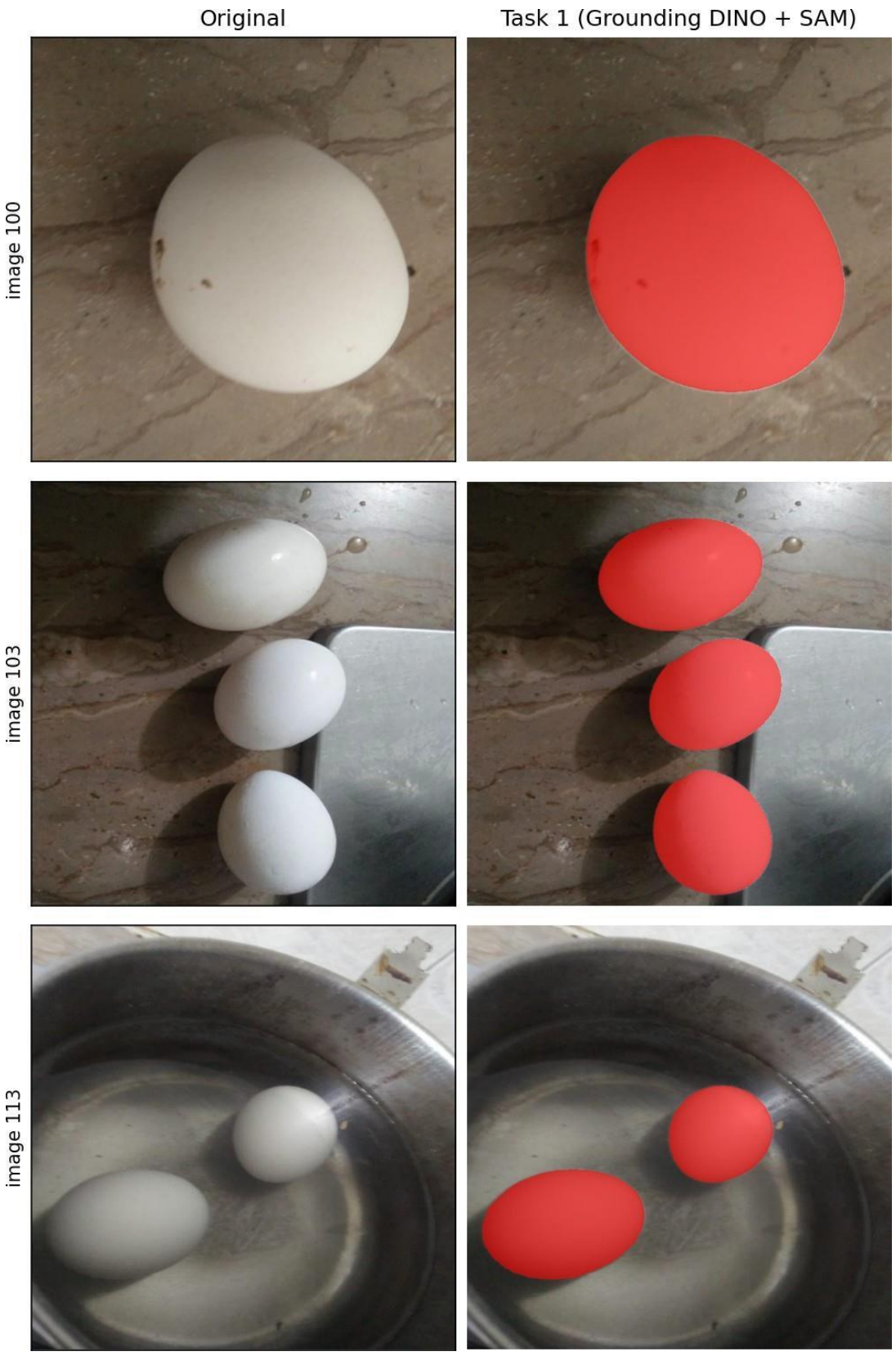

Original
Task 1 (Grounding DINO + SAM)
image 100
image 103
image 113



Figure 1. Original image (left) and Task 1 segmentation (right) for six representative images.

Image 22 has the smallest mask (3% of the image), so I checked it against the original. The outline in Figure 2 fits the egg edge, so it is a correct detection and not a false one.



Figure 2. Image 22, the smallest mask: original (left) and mask outline in red (right).

7. Failures and imperfect cases

I did not find a complete failure (a missed egg or a false egg) in the images I inspected. One case was imperfect:

- **Image 10 (egg under leaves, Figure 1):** the egg is partly covered by leaves, and the mask has a notch and a ragged edge where the stems cross it. The mask also does not extend to the part of the egg hidden behind the leaves at the top right. This is expected for any segmentation method, since the leaves really do occlude the egg.

8. Limitations

- **The masks are model predictions, not human annotations.** Any systematic bias in SAM's boundaries (for example around shadows or occluded parts) becomes part of the "ground truth" in Task 3, so the Task 3 scores measure agreement with SAM and not absolute accuracy.
- **My visual check did not cover all 110 images.** I reviewed the first 12 overlays and the 12 flagged images. Rare errors in the remaining images could still exist.
- **The low-confidence images were checked by eye only.** I did not compute a numeric mask-quality metric in Task 1.
- **Hand-set thresholds and filter.** The box threshold, text threshold and the 85% area filter were kept at their starting values. They worked on this dataset, but I have not tested how sensitive the results are to them.

- **Two models in series.** If Grounding DINO misses an egg, SAM never sees it. Pale eggs on pale backgrounds are the most likely case, and the dataset contains several of these.
- **Overlap rule.** Where two masks overlap, the lower-numbered egg keeps the pixels, which is arbitrary. This only affects a thin boundary between touching eggs.
- **Compute.** The pipeline needs a GPU for practical speed and downloads about 2 GB of model weights on first run.

9. Reproducibility

```
pip install torch transformers pillow opencv-python numpy
python task1_sam_segment.py --images data --out outputs/task1
```

Outputs: `outputs/task1/masks/` (instance PNGs), `outputs/task1/overlays/` (red overlays), `outputs/task1/results.json` (boxes and scores). The figure `task1_examples.png` shows six representative cases: a single egg, multiple eggs, the pan, touching eggs, leaf occlusion, and a white egg on a white label.

10. AI tool disclosure

I used Claude (Anthropic) as a coding assistant for this task: to compare approaches, to draft the segmentation script, and to write the inspection snippets. I ran everything myself in Colab, looked at the outputs, and made the decisions about what to keep, as described above.

Task 2: Segmentation Using Classical Image Processing

1. Objective

Task 2 asks for an egg segmentation algorithm built only from classical image processing, tuned to match the Task 1 masks (Grounding DINO + SAM), which I treat as ground truth. This section documents (a) a leakage-safe 4:1 dev/eval split, (b) the metric and the rule I used to accept or reject changes, (c) every trial I ran, including the failed ones, and (d) the final algorithm. All tuning used the 88 dev images. The 22 eval images were scored once, at the end, with the frozen pipeline (section 8); the failure analysis follows in Task 3.

2. Dev/eval split and leakage control

Duplicate check. All 110 file ids are unique, so there are no augmented copies by name. But many photos share the same scene (floor, tiles, steel pan, white carry bag, plate). A random split could put near-identical scenes on both sides and inflate the eval score.

Grouping by scene. I painted each egg with the mean background colour using the Task 1 masks, so the comparison depends on the background and not on the egg. I embedded the images with ResNet-18 and clustered them (agglomerative, cosine distance, average linkage, threshold 0.25). This gave 20 scene groups (the largest have 24, 13, 12, 10 and 7 images; three are singletons). A visual check of the six largest groups showed clean scenes for hands, steel pan, white bag and keyboard, and coarser merges for floors and tiles. Coarse groups are the safe direction for leakage.

Split attempt 1 (rejected). I assigned whole groups to eval, with eval groups of at most 10 images, at least 4 groups, and a similar share of multi-egg images. It gave 88/22 with 0 shared groups, but none of the 17 multi-egg images landed in eval: 12 of them sit in the 24-image group (too big for eval) and 5 in the 12-image steel-pan group. An eval set without multiple or touching eggs would hide the hardest case, so I rejected it.

Split attempt 2 (used). I forced the steel-pan group (12 images, 5 multi-egg) into eval and filled the other 10 images with small groups of at most 4 images, choosing the combination with the most different scenes. Result: 88 dev and 22 eval images, eval groups 5, 10, 11, 14, 16, 17 and 18, 0 groups in both splits, and multi-egg images split 12 in dev and 5 in eval.

Limitations of the split. 12 of the 22 eval images (55%) come from one scene (the steel pan), and the multi-egg share in eval is 23% against 15% overall, so eval has more multi-egg images than average and leans on pan scenes, which scored higher than the rest of eval (section 8). The clustering threshold was set by eye. I looked at the Task 1 masks of 8 hard eval images (bubbling water, egg tray, backlit hand) to sanity-check the ground truth,

but I did not run or tune any classical method on eval until the final, single run with the frozen pipeline.

3. Metric and decision rule

- Per-image IoU between my predicted mask and the Task 1 mask, both taken as the union of all eggs in the image. I report the mean, the median, and the number of failures (IoU below 0.5) over the 88 dev images.
- The union IoU does not penalise merging touching eggs. Instance-level metrics are part of Task 3.
- **Rule, fixed before the first trial:** keep a change only if the mean dev IoU rises by at least 0.005 over the best kept trial and the number of failures does not go up. One change per trial. The change and the reason are written before looking at the score.
- Every trial is saved to experiment_log.csv (with its parameters), and the per-image scores to per_image_iou.csv.

4. Overview of all trials

Trial	Change	Mean IoU	Median	Fails (<0.5)	Δ mean vs previous kept	Decision
T0	Grayscale + Otsu baseline	0.379	0.278	61	–	KEPT
T1	Lab colour distance from border colour, then Otsu	0.359	0.285	57	−0.020	DISCARDED
T2	T0 + size, border and solidity filter	0.217	0.000	66	−0.162	DISCARDED
T3	24-level threshold sweep, ellipse-fit + edge scoring	0.509	0.535	40	+0.131	KEPT
T4	+ candidates from Lab a, Lab b and saturation	0.564	0.571	34	+0.055	KEPT
T5	Brightness added to the ranking	0.558	0.568	37	−0.006	DISCARDED
T6	Brightness gate (at least 0.6) on acceptance	0.590	0.694	34	+0.026	KEPT
T7	+ fallback when the output is empty	0.598	0.694	33	+0.007	KEPT
T8	Blobs replaced by their fitted ellipse	0.589	0.686	33	−0.008	DISCARDED
T9	+ GrabCut per blob (ring 0.4)	0.671	0.769	21	+0.073	KEPT
T9a	GrabCut ring 0.2	0.642	0.744	24	−0.029	DISCARDED
T9c	GrabCut ring 0.6	0.682	0.766	18	+0.011	KEPT

Trial	Change	Mean IoU	Median	Fails (<0.5)	Δ mean vs previous kept	Decision
T9d	GrabCut ring 0.8	0.688	0.779	18	+0.006	KEPT
T9e	GrabCut ring 1.0	0.688	0.793	19	+0.0003	DISCARDED
T10	Accept threshold lowered from 0.9 to 0.85	0.698	0.784	20	+0.010	DISCARDED
T11	Single-egg retry at accept score 0.85 when the 0.9 pass is empty	0.719	0.800	16	+0.031	KEPT
Frozen	T11 re-run with a fixed GrabCut seed (not a tuning trial; official dev number; final pipeline)	0.714	0.807	17	–	KEPT

Notes: T9a and T9c are compared with T9, T9d with T9c, and T9e, T10 and T11 with T9d. There is no trial T9b in the log. The vs_best_kept column in experiment_log.csv for T7 to T9e was filled in before the earlier decisions were recorded, so it shows other numbers; the Δ column above is recomputed against the previous kept trial.

5. Experiment log

T0: Baseline (KEPT)

- **Change:** grayscale, Gaussian blur, Otsu threshold, remove blobs under 1% of the image, fill holes.
- **Why:** the simplest classical method, as a reference for every later trial.
- **Parameters:** blur 5, minimum blob area 1% of the image.
- **Dev result:** mean IoU 0.379, median 0.278, 61 of 88 below 0.5.
- **Decision:** kept as the baseline. The worst cases were white eggs on white or bright backgrounds (bag, wall, tiles), dark backlit eggs (20, 44) where Otsu picks the bright wall, and textured backgrounds (plate, hand) that split into fragments.

T1: Colour distance from the background (DISCARDED)

- **Change:** replaced grayscale with the Lab colour distance from the median colour of a thin border strip, then Otsu.
- **Why:** T0 assumes the egg is brighter than the background, which fails on dark backlit eggs (20, 44).
- **Parameters:** blur 5, border strip 6% of the image, minimum blob area 1%.
- **Dev result:** mean 0.359, median 0.285, 57 below 0.5 (−0.020).

- **Decision:** discarded. White eggs on a white bag, wall or cloth are close to the background colour, so the method selected dark corners and holes instead (IoU 0.00 on 30, 106, 104, 112, 111, 32, 6 and 5).

Diagnostic D1 (not scored). I measured the 111 eggs in the dev masks: egg area is 2.7–38% of the image (median 9.9%), centres cluster near (0.50, 0.45), median solidity is about 1.0, and only 1 of 111 eggs touches the image border.

T2: Size, border and shape filter (DISCARDED)

- **Change:** T0 plus a filter that keeps only regions covering 2–40% of the image, not touching the border, with solidity of at least 0.9.
- **Why:** D1 priors, and T0 failures were mostly large border-touching background regions.
- **Parameters:** blur 5, area 2–40%, solidity at least 0.9, border rejection on.
- **Dev result:** mean 0.217, median 0.0, 66 below 0.5, and an empty output on 62 of 88 images (−0.162).
- **Decision:** discarded. I had predicted a modest gain and was wrong: Otsu usually merges the egg with a large border-touching region, so the filter removed the egg together with the background. A post-filter cannot repair a bad threshold.

T3: Threshold sweep with shape scoring (KEPT)

- **Change:** replaced the single Otsu threshold with a sweep of 24 grey-level thresholds, each applied on the bright side and on the dark side. After hole filling and a 5×5 opening, candidates must cover 2–40% of the image and not touch the border. Each is scored by the IoU with its best-fit ellipse plus 0.5 × the edge strength along its boundary. Candidates scoring at least 0.9 are accepted (up to 3 eggs, overlapping ones dropped).
- **Why:** T2 showed that one threshold cannot isolate the egg. Some threshold in a sweep may, and an ellipse fit can pick that one.
- **Parameters:** blur 5, 24 thresholds from 20 to 235, area 2–40%, edge weight 0.5, accept score 0.9, at most 3 eggs, overlap limit 0.3.
- **Dev result:** mean 0.509, median 0.535, 40 below 0.5 (+0.131); 13 images at exactly 0.
- **Decision:** kept. Remaining errors: dark holes and shadows in the white bag scored higher than white eggs, and ash-covered eggs gave an empty output.

Diagnostic D2 (not scored). For every dev egg I found the candidate from the T3 sweep that matches its mask best. With grey-level candidates the mean best IoU was only 0.638, and just 60% of eggs had a candidate with IoU of 0.6 or more. So candidate generation, not ranking, was the main limit.

T4: Colour-channel candidates (KEPT)

- **Change:** T3 plus extra candidates from Lab a, Lab b and HSV saturation (12 thresholds each, bright and dark side). Scoring and filters unchanged.
- **Why:** D2. White eggs on white backgrounds may differ from the background in colour even when the brightness matches.
- **Parameters:** as T3, plus 12 thresholds per colour channel.
- **Dev result:** mean 0.564, median 0.571, 34 below 0.5 (+0.055); images at exactly 0 fell from 13 to 8. The best-candidate ceiling rose from 0.638 to 0.715, and 69% of eggs now have a candidate with IoU of 0.6 or more (was 60%).
- **Decision:** kept. In the white-bag images (125, 106, 104, 111, 123) the output was still a dark hole instead of the egg.

T5: Brightness in the ranking (DISCARDED)

- **Change:** candidates and acceptance test as in T4; ranking = ellipse fit + 0.5 × edge + 0.5 × mean brightness inside the blob.
- **Why:** in 5 of T4's 8 worst images a dark hole in the white bag was output instead of the white egg, and I believed the egg was ranked below it.
- **Parameters:** edge weight 0.5, brightness weight 0.5, accept score 0.9, at most 3 eggs, overlap limit 0.3.
- **Dev result:** mean 0.558, median 0.568, 37 below 0.5 (−0.006); 1 image improved by more than 0.1 and 3 got worse by more than 0.1.
- **Decision:** discarded, and my diagnosis was wrong. The output is the union of all accepted candidates, so reordering them cannot remove a false blob; ranking only matters when the cap of 3 is reached.

Diagnostic D3 (not scored). I computed ellipse fit, edge strength and brightness for every candidate and compared 513 good-egg candidates (IoU at least 0.8) with 191 accepted false blobs. Only 1 of the 513 good candidates fell below the 0.9 acceptance score, so acceptance was not blocking real eggs. Brightness separated the groups cleanly: good eggs had 0.70, 0.77 and 0.82 at the 10th, 50th and 90th percentile, false blobs 0.14, 0.33 and 0.55. Ellipse fit also separated them (median 0.95 against 0.74), but edge strength did not (medians 0.61 against 0.56, and false blobs had the higher 90th percentile, because dark holes have sharp edges). In the white-bag images the best egg candidate was poor (IoU 0.00–0.73) and the egg edge was faint (about 0.1).

T6: Brightness gate on acceptance (KEPT)

- **Change:** T4 candidates and scoring unchanged; a candidate is accepted only if its mean brightness inside is at least 0.6.

- **Why:** D3. Brightness had only been used for ordering in T5, which cannot remove blobs from the union. As a gate it should remove the dark false blobs.
- **Parameters:** edge weight 0.5, brightness gate 0.6 (chosen from the D3 distributions, between 0.55 and 0.70), accept score 0.9, at most 3 eggs, overlap limit 0.3.
- **Dev result:** mean 0.590, median 0.694, 34 below 0.5 (+0.026). 18 images improved by more than 0.1 and 5 got worse by more than 0.1.
- **Decision:** kept. The gate removed dark false blobs but also zeroed some real, darker eggs (44, 78): 11 images gave an empty output. This encodes the assumption that eggs are pale under normal lighting.

T7: Fallback when the output is empty (KEPT)

- **Change:** T6 plus a fallback. If nothing is accepted, retry with a brightness gate of 0.3 and keep only the single best candidate.
- **Why:** T6 left 11 empty outputs, and every image contains an egg; 78, 27, 44 and 20 are real eggs darker than the 0.6 gate.
- **Parameters:** as T6, plus fallback brightness gate 0.3.
- **Dev result:** mean 0.598, median 0.694, 33 below 0.5 (+0.007). Empty outputs fell from 11 to 4.
- **Decision:** kept, with a small gain. Only image 78 recovered above 0.5 (0.64); the fallback picked a wrong blob in most other cases.

T8: Replace blobs by fitted ellipses (DISCARDED)

- **Change:** T7, but each accepted blob is replaced by its filled fitted ellipse.
- **Why:** on 160 and 50 the T6 outputs were clean but partial (precision about 0.97, recall 0.39–0.45), and eggs are nearly elliptical (median ellipse fit 0.95).
- **Parameters:** as T7, plus ellipse filling on.
- **Dev result:** mean 0.589, median 0.686, 33 below 0.5 (−0.008). No image improved by more than 0.1, and one got worse by more than 0.1 (image 12, −0.14).
- **Decision:** discarded. Ellipse completion did not recover 160 (0.44 to 0.42) or 50 (0.39 to 0.38).

Diagnostic D4. In the failing images no global-threshold candidate covers the whole egg, but partial blobs exist inside the egg (104, 111, 149, 160, 50). Such a partial blob can seed a colour-model refinement.

T9: GrabCut refinement of each accepted blob (KEPT)

- **Change:** T7 plus GrabCut run on each accepted blob. Sure foreground = the blob eroded; probable foreground = the rest of the blob; probable background = a ring around the blob (0.4 × its radius); sure background beyond that. 10 iterations.
- **Why:** D4. The blob only has to lie inside the egg; GrabCut can grow it using the egg's own colour model.
- **Parameters:** ring 0.4, 10 iterations.
- **Dev result:** mean 0.671, median 0.769, 21 below 0.5 (+0.073). 32 images improved by more than 0.1 and none got worse by more than 0.1.
- **Decision:** kept; this was the largest single gain after T3. Partial eggs were recovered (160: 0.44 to 0.59, 50: 0.39 to 0.53). It does not help where the seed is wrong or missing (104, 111 and 149 stayed at 0).

T9a, T9c, T9d, T9e: GrabCut ring width

The ring width is a hand-set parameter, so I swept it, changing only the ring.

Trial	Ring	Mean IoU	Median	Fails	Δ vs previous kept	Decision and reason
T9a	0.2	0.642	0.744	24	−0.029 (vs T9)	DISCARDED. A narrow ring gives GrabCut too little background to model
T9c	0.6	0.682	0.766	18	+0.011 (vs T9)	KEPT. A wider ring is better; new base for later trials
T9d	0.8	0.688	0.779	18	+0.006 (vs T9c)	KEPT. The gain is shrinking, so the sweep has nearly plateaued; this is the final ring value
T9e	1.0	0.688	0.793	19	+0.0003 (vs T9d)	DISCARDED. Below the 0.005 bar and one more failure; plateau reached

T10: Lower the acceptance threshold (DISCARDED)

- **Change:** T9d with the accept score lowered from 0.9 to 0.85.
- **Why:** D4. Images 111 and 149 have a partial candidate with a score of 0.88, just under 0.9, and GrabCut might refine a weaker seed.
- **Parameters:** ring 0.8, 10 iterations, accept score 0.85.
- **Dev result:** mean 0.698, median 0.784, 20 below 0.5 (+0.010 mean, but failures rose from 18 to 20).
- **Decision:** discarded, because the pre-set rule needs the failure count not to go up. The idea worked where I expected: 111 went from 0.0 to 0.98 and 149 from 0.0 to 0.76. But 7 images got worse by more than 0.1. In 28 and 130 the precision fell (0.99 to 0.29 and 0.93 to 0.46) and the predicted area grew 3–4 times (28: 4.2% to 14.5% against 5.7% in the

ground truth; 130: 12.7% to 25.7% against 11.8%). The lower threshold admitted an extra false blob next to an egg that had already been found.

T11: Single-egg retry at a lower accept score (KEPT)

- **Change:** T9d, plus: if nothing is accepted at score 0.9 (brightness gate 0.6), retry once at score 0.85 and keep a single egg, before the gate-0.3 fallback.
- **Why:** T10 recovered 111 and 149 but added a false blob in 28 and 130, where an egg had already been accepted at 0.9. Using 0.85 only on images with an empty output should keep the gain and avoid the loss.
- **Parameters:** accept score 0.9, retry score 0.85, brightness gate 0.6, fallback gate 0.3, ring 0.8, 10 iterations.
- **Dev result:** mean 0.719, median 0.800, 16 below 0.5 (+0.031). 5 images improved by more than 0.1 (111: 0.0 to 0.98, 149: 0.0 to 0.76, 28: 0.73 to 0.99) and none got worse by more than 0.1 (worst: image 37, −0.07). Images 5 and 104 stay at 0.0.
- **Decision:** kept. It keeps the T10 gains without T10's extra false blobs. A sanity run with the retry score set back to 0.9 gave a mean of 0.6892 against 0.6880 for T9d; the small difference comes from GrabCut's random initialisation (next entry).

Frozen pipeline: fixed GrabCut seed (KEPT, not a tuning trial)

- **Change:** none to the algorithm. The OpenCV random seed is set to 0 before each image, so repeated runs give identical masks (checked on 20 images).
- **Why:** GrabCut uses random initialisation. The same pipeline gave 0.6880 and 0.6892 (T9d) and 0.719 and 0.7142 (T11) on dev. The seed had to be fixed before the single eval run so that the reported numbers can be reproduced.
- **Dev result:** mean 0.714, median 0.807, 17 below 0.5. This is the official dev number.
- **Decision:** kept as the frozen final pipeline. No parameter was changed after T11.

6. How the experiments led to the algorithm

The baseline (T0) failed because a single global threshold cannot separate a white egg from a bright or textured background, and the backlit eggs break its brightness assumption. Colour distance from the border (T1) and a shape filter on the Otsu output (T2) both made things worse, which told me the real problem was how candidate regions were produced, not how they were filtered. T3 generated many candidates with a threshold sweep and chose among them by shape, and T4 added colour channels after the oracle check (D2) showed the candidates were the limit. T5 failed because I misjudged how the union output works, and the diagnostic that followed (D3) showed that brightness works as an acceptance gate, which T6 used. T7 recovered a few dark

eggs. T8 showed that filling the blob with an ellipse does not fix partial masks. D4 showed that partial blobs still sit inside the egg, and T9 used them as GrabCut seeds, which gave the biggest gain. The ring sweep (T9a to T9e) tuned that step until it plateaued at 0.8. T10 showed a real trade-off: a lower accept threshold recovers some eggs but lets false blobs into images that were already correct, and it broke my own rule, so I did not keep it.

T11 then kept the part of T10 that worked. It applies the lower accept score only when the normal pass finds nothing, which recovered 111 and 149 and broke no image by more than 0.1. After that I fixed GrabCut's random seed, froze the pipeline, and ran it once on the eval split.

7. Final algorithm (T11 configuration, frozen)

1. **Candidates.** Blur the image (5×5). From the grey channel, take 24 thresholds between 20 and 235; from Lab a, Lab b and HSV saturation (each rescaled to 0–255), take 12 thresholds each. Use every threshold on the bright side and on the dark side. For each mask, fill holes, apply a 5×5 opening, and keep connected components that cover 2–40% of the image and do not touch the border.
2. **Score.** For each candidate: IoU with its best-fit ellipse + 0.5 × mean edge strength along its contour (Sobel gradient magnitude scaled by its 99th percentile).
3. **Accept.** Go through candidates from best to worst; accept those with score at least 0.9 and mean grey brightness inside of at least 0.6, up to 3 eggs, skipping any candidate that overlaps an accepted one by more than 30% of the smaller area.
4. **Fallback.** If nothing was accepted, first retry once at accept score 0.85 (brightness gate still 0.6) and keep the single best candidate. If that also finds nothing, retry with a brightness gate of 0.3 at score 0.9 and keep the single best candidate.
5. **Refine.** Run GrabCut (10 iterations) on each accepted blob: sure foreground = the blob eroded with a kernel of about a quarter of its radius; probable foreground = the rest of the blob; probable background = a ring extending 0.8 × the radius around the blob; sure background beyond it. If the eroded core has fewer than 50 pixels, or GrabCut fails, the blob is kept as it is.
6. **Output.** The union of the refined masks.

GrabCut uses random initialisation, so the OpenCV random seed is fixed (set to 0 before each image) to make repeated runs identical.

8. Final results on dev and eval

Pipeline	Split	Mean IoU	Median IoU	Images below 0.5
Baseline (T0)	dev (88)	0.379	0.278	61
T9d (before the retry step)	dev (88)	0.688	0.779	18
Final frozen pipeline (T11, fixed seed)	dev (88)	0.714	0.807	17
Final frozen pipeline (T11, fixed seed)	eval (22)	0.634	0.670	6

The eval split was scored once, with the frozen pipeline, after all tuning was finished. Mean IoU is 0.634 (median 0.670), with 6 of 22 images below 0.5 and one empty output, against 0.714 on dev (median 0.807, 17 of 88 below 0.5).

- Without the three egg-tray images whose Task 1 masks are ambiguous (42, 116, 117), the eval mean is 0.669.
- The steel-pan scene (12 images) averages 0.731; the other 10 eval images average 0.517.
- Three images score exactly 0: 42 (egg tray, ambiguous ground truth), 22 (white egg on a white label) and 24 (dark backlit egg). The other images below 0.5 are 126 (0.38), 62 (0.40) and 135 (0.47). The causes are analysed in Task 3.

With only 22 images, one image moves the mean by about 0.045, so the dev-to-eval gap is indicative and not precise. The T11 run before the seed was fixed gave 0.719 on dev with 16 failures; the official dev number is the seeded 0.714.

How I froze the pipeline and ran eval

1. **Stop tuning.** T11 was the last scored trial. No parameter was changed after it.
2. **Make runs reproducible.** GrabCut's random initialisation made two runs of the same pipeline differ (T9d: 0.6880 and 0.6892; T11: 0.719 and 0.7142). I fixed the OpenCV random seed (0) and checked that two repeat runs on 20 dev images gave identical masks. The seeded dev run (mean 0.714, 17 failures) is the official dev number.
3. **Run eval once.** I loaded the 22 eval images, confirmed that none of them is in dev (overlap 0), and ran the frozen pipeline a single time. The predicted masks are saved as one PNG per image (outputs/task2_pred) and the per-image IoU in eval_results.csv.
4. **No tuning on eval.** The eval numbers are not used to change the algorithm. Improvements suggested by the eval failures are listed as suggestions in Task 3 and are not applied to the reported result.

What the eval predictions look like

From the saved masks (overlays against the Task 1 masks follow in Task 3):

- 8 of the 22 images score 0.8 or higher, 6 score between 0.6 and 0.8, and 8 score below 0.6.
- Image 42 is the only empty output.
- Several masks contain small isolated fragments left by GrabCut. Image 22 has 17 separate components (one of 18,271 pixels, one of 901 pixels and 15 specks of 34 pixels or less), image 153 has 7, and images 114 and 159 have 5 each. For the instance-level evaluation in Task 3, predicted components under 1% of the image (4,096 pixels) are ignored; the smallest dev egg covers 2.7%, so no real egg is removed.
- Images 129 and 153 appear to contain an extra blob next to the eggs (129 shows two eggs and 153 one), 22 and 24 give an irregular mask that does not look like an egg, and 62 and 159 have a thin tail or streak attached to the egg.

9. Failures and limitations

- **Pale-egg assumption.** The brightness gate (0.6) and the fallback gate (0.3) were set by hand from dev statistics, and I did not sweep them. A brown or dark egg will only be found through the fallback, and often not at all.
- **Priors from dev.** Area 2–40%, no contact with the border (1 of 111 dev eggs touches it) and at most 3 eggs are assumptions of this dataset.
- **No seed, no refinement.** GrabCut can only refine an egg that already has a seed. Where the seed is missing or wrong, it cannot help; images 5 and 104 are still at IoU 0.0 with the final configuration.
- **False blobs next to a correct egg.** The accept-and-union design means one extra accepted blob lowers the IoU of an otherwise correct image (the T10 result on 28 and 130).
- **Union mask.** Touching eggs are not separated, and the metric does not penalise that. Instance-level evaluation comes in Task 3.
- **Ground truth is SAM, not human.** Some eval images (the egg-tray images 42, 116, 117) have ambiguous Task 1 masks, so a disagreement there is not automatically a failure of my method.
- **Dev scores are optimistic.** The algorithm and its thresholds were chosen using the 88 dev images, so the eval score was expected to be lower, and it was (section 8).
- **Remaining dev failures.** 17 of 88 dev images are still below IoU 0.5 with the final pipeline.
- **Compute.** About 120 threshold masks per image plus ellipse fitting and GrabCut; building the candidates for 88 images takes a few minutes on a Colab CPU.

- **Run-to-run noise.** GrabCut's random initialisation moves the mean IoU by about 0.005 and the failure count by about 1 between runs of the same pipeline (T9d: 0.6880 and 0.6892; T11: 0.719 and 0.7142). Differences of that size in the ring sweep (T9c to T9e) are within noise, so ring values from 0.6 to 1.0 behave about the same and 0.8 is a reasonable choice, not a proven optimum. The T9 gain (+0.073) and the T11 gain (+0.031) are well above the noise.
- **Dev-to-eval gap.** Eval scores lower than dev (mean 0.634 against 0.714, median 0.670 against 0.807, 6 of 22 images below 0.5 against 17 of 88). This is expected because the thresholds and the design were chosen on dev, and eval contains harder scenes (egg tray, backlit hand, white egg on a white label). With 22 images the estimate is rough.

10. Reproducibility

Files: egg_lib.py (all functions; the final one is seg_final, which calls seg_gc2 with ring=0.8 and a fixed GrabCut seed), split.csv (the dev/eval assignment and scene groups), experiment_log.csv and per_image_iou.csv (every trial and per-image dev scores), eval_results.csv and outputs/task2_pred (eval IoU per image and the predicted masks). To rerun: restore data/ and outputs/task1/masks, load egg_lib.py, then call run_trial with seg_gc and the parameters listed for each trial.

Colab sessions reset without warning, so I kept the dataset zip, the Task 1 masks, egg_lib.py, the split and the logs in Google Drive and restored them at the start of each session.

11. AI tool disclosure

I used Claude (Anthropic) as a coding and analysis assistant for Task 2: to propose trial ideas, to draft the scoring and logging code, and to help read the diagnostics. I ran every trial myself in Colab, looked at the outputs, and made the keep or discard decisions using the rule in section 3.

Task 3: Evaluation and Analysis

1. Setup

Ground truth: the Task 1 masks (Grounding DINO + SAM). Predictions: the output of my frozen Task 2 pipeline (T11 configuration with a fixed GrabCut seed) on the 22 eval images, which were not used for tuning and were scored once. For the 88 dev images I only kept the per-image IoU (the predictions were not saved), so dev appears only in the IoU comparison.

Metrics

- **Pixel level** (all eggs of an image merged into one mask): IoU, Dice, precision (share of predicted pixels that are egg), recall (share of egg pixels found), boundary F1 with a 3-pixel tolerance on 1-pixel contours, and area ratio (predicted area divided by egg area).
- **Instance level:** each Task 1 egg is matched one-to-one (greedy, highest IoU first) to a predicted connected component with an IoU of at least 0.5. Unmatched Task 1 eggs are false negatives (FN) and unmatched predicted components false positives (FP). Predicted components under 4,096 pixels (1% of the image) are ignored: GrabCut leaves isolated specks (65 raw components, 28 after this rule), and the smallest dev egg covers 2.7% of the image. I set this rule before computing the instance metrics.
- **Failure type** (a heuristic used only for sorting): empty; wrong object (recall under 0.2); partial egg (precision at least 0.7 and recall under 0.6); leak (recall at least 0.6 and precision under 0.6).

2. Quantitative results

Pixel-level metrics on the eval split (22 images)

Metric	Mean	Median
IoU	0.634	0.669
Dice	0.718	0.802
Precision	0.763	0.969
Recall	0.723	0.879
Boundary F1 (3 px)	0.528	0.551

Instance-level metrics on the eval split

Task 1 eggs	Predicted components (raw / after size rule)	TP	FP	FN	Precision	Recall	F1
27	65 / 28	21	7	6	0.750	0.778	0.764

The egg count is correct in 19 of 22 images. It is wrong in 42 (0 instead of 1), 153 (2 instead of 1) and 129 (3 instead of 2).

Dev against eval (IoU only)

	Dev (88 images)	Eval (22 images)
Mean IoU	0.714	0.634
Median IoU	0.807	0.669
Images with IoU at least 0.8	51%	36%
Images with IoU below 0.5	19% (17)	27% (6)

What the numbers say

- Most masks are good and a few images pull the mean down: the median precision is 0.969 against a mean of 0.763, and the median recall is 0.879 against a mean of 0.723.
- On eval, 8 images have IoU of at least 0.8, 6 are between 0.6 and 0.8, 5 are between 0 and 0.6, and 3 are exactly 0 (42, 22 and 24). The other 19 images average 0.734, which is close to the dev mean of 0.714.
- The boundary F1 is low (0.53) even though the IoU is decent, so the outlines often differ from the Task 1 outlines by more than 3 pixels. I did not check why.
- The eval split contains no touching eggs. The four multi-egg images (113, 114, 128, 147) all have the right egg count, so the weakness of the union mask on touching eggs is not tested here.
- By scene, the steel pan averages 0.731 (12 images) and the other scenes 0.517 (10 images). Without the three zero images the other scenes average 0.74, so the difference comes from those three images and not from the scenes being harder in general.
- IoU correlates with the contrast between the egg and its surroundings (0.60) and with the egg size (0.42), but with 22 images this is only a rough indication.

All eval images, sorted by IoU

Image	Scene group	IoU	Precision	Recall	Main issue
42	16	0.00	0.00	0.00	empty output
22	10	0.00	0.00	0.00	wrong object
24	14	0.00	0.00	0.00	wrong object
126	5	0.38	0.99	0.38	part of the egg missed
62	11	0.40	0.40	1.00	extra region
135	5	0.47	0.97	0.47	part of the egg missed
153	5	0.56	0.57	0.96	extra region
117	16	0.58	0.91	0.61	part of the egg missed
129	5	0.62	0.68	0.89	extra region
107	5	0.66	1.00	0.66	part of the egg missed
116	17	0.66	1.00	0.66	part of the egg missed
145	5	0.68	0.74	0.89	extra region
159	5	0.74	0.74	1.00	extra region (from P and R only)
52	11	0.76	0.99	0.77	slightly partial
16	11	0.84	0.98	0.85	good
124	5	0.85	0.88	0.96	good
113	5	0.86	0.99	0.87	good
39	10	0.96	0.99	0.97	good
49	18	0.97	0.97	1.00	good
147	5	0.97	1.00	0.98	good
128	5	0.98	0.99	0.99	good
114	5	0.99	0.99	1.00	good

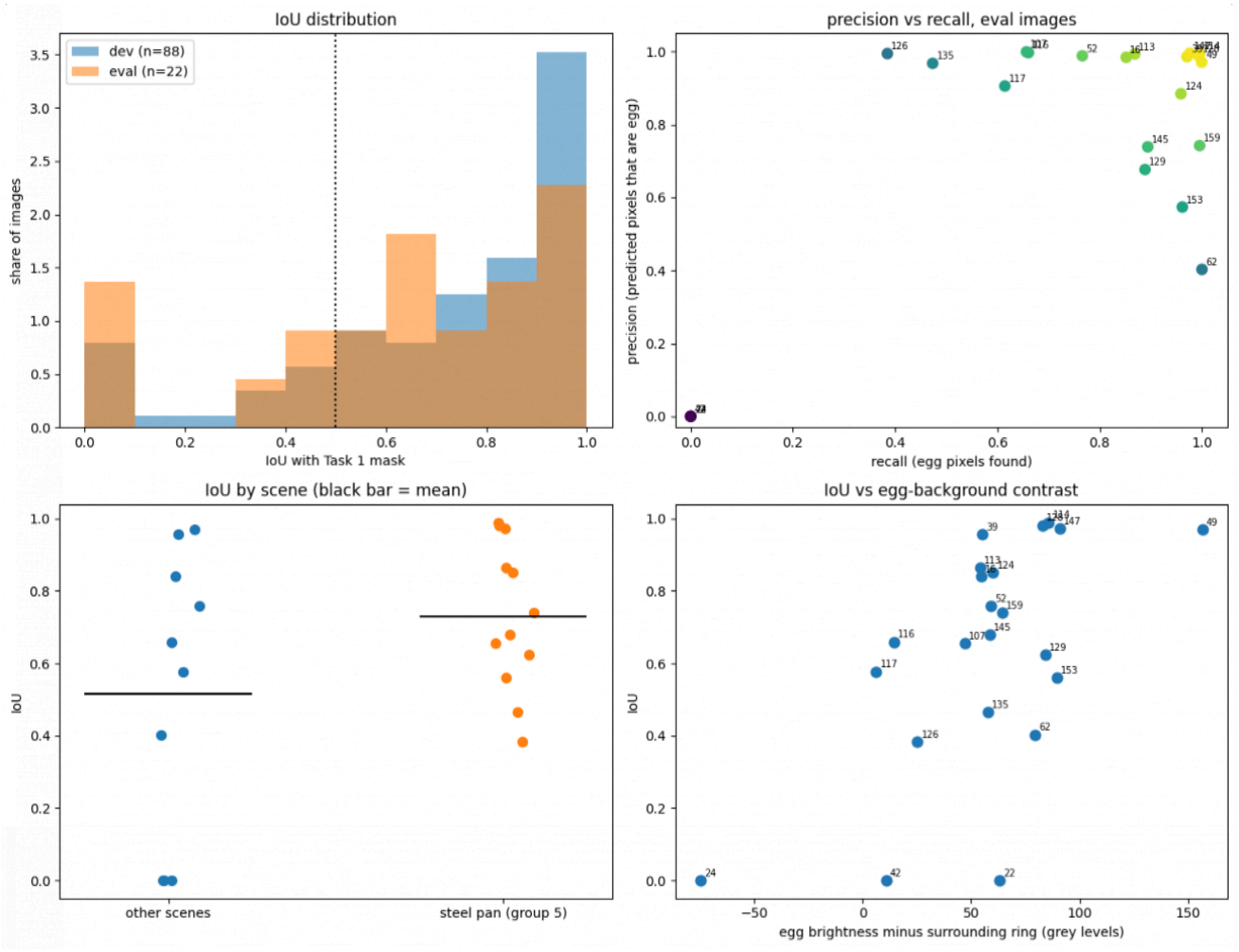


Figure 1. Top left: IoU distribution on dev (88 images) and eval (22 images). Top right: precision against recall for each eval image (colour = IoU). Bottom left: IoU by scene (the black bar is the mean). Bottom right: IoU against the contrast between the egg and its surroundings.

3. Qualitative analysis

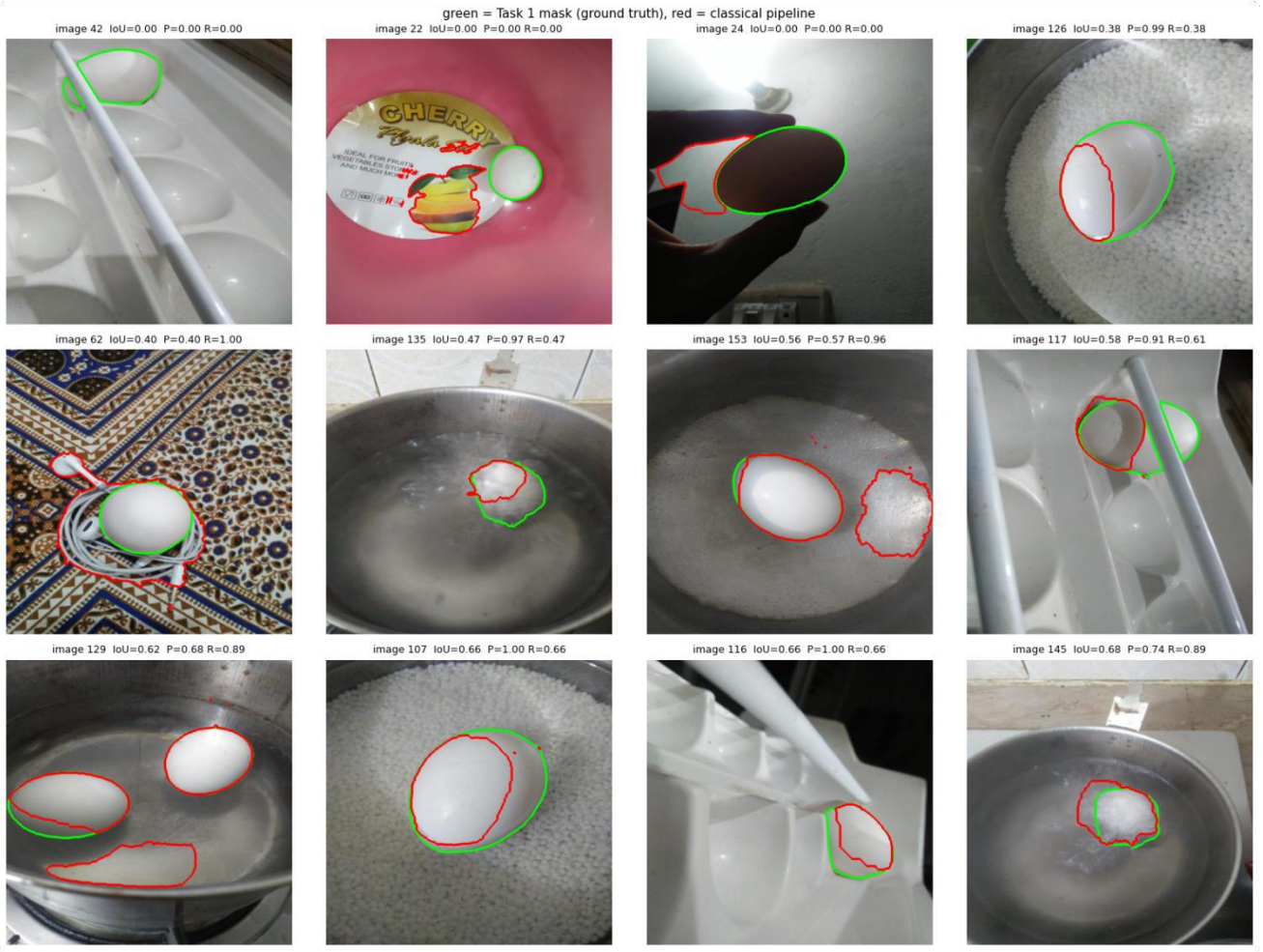


Figure 2. The 12 eval images with the lowest IoU. Green: Task 1 mask (ground truth). Red: output of the classical pipeline.

I looked at the overlays of the 12 images with the lowest IoU (all at 0.68 or below). The other 10 images (IoU of at least 0.74) I only know from the numbers. What I describe below is what the overlays show; the causes are my hypotheses and I did not test them.

3.1 Total failures (IoU 0): images 42, 22, 24

- **Image 22:** the output is the yellow and orange fruit picture on the label, plus small marks from the printed text, and not the white egg next to the label. The egg is white and low in colour, the picture is strongly coloured; my pipeline checks brightness and shape but not colour.
- **Image 24:** the egg is dark, held in front of a bright light, and the only egg in eval that is darker than its surroundings (contrast of about −75 grey levels). The output is the dark hand region next to the egg. My pipeline expects pale eggs (brightness gate 0.6, fallback gate 0.3), which is the likely reason, but I did not verify it.

- **Image 42:** the output is empty. The Task 1 mask itself is ambiguous here, an egg-shaped cavity in a white egg tray.

3.2 An extra bright region is merged or added: images 62, 153, 129, 145

- **Image 62:** the egg is found completely (recall 1.00), but the outline also contains the white earphone cable around it, so the precision is 0.40.
- **Image 153:** the egg is correct, and a second, irregular blob on the foam at the right edge of the pan is accepted as an egg (precision 0.57, two components instead of one).
- **Image 129:** both eggs are correct, plus a flat bright patch at the bottom of the pan (precision 0.68, three components instead of two).
- **Image 145:** the outline extends from the egg into the bubbles around it (precision 0.74).
- Common cause (hypothesis): the pipeline takes the union of every accepted blob, so one extra accepted region lowers the IoU of an otherwise correct image.

3.3 Part of the egg is missed: images 126, 107, 117, 116, 135

- **Images 126 and 107** (eggs in sago grains): the output covers the lit side of the egg and stops with a clean edge where the shading starts, so the shaded side is missing (recall 0.38 and 0.66, precision about 1).
- **Image 117:** the output covers the main part of the egg and misses the part on the other side of the white bar of the tray (recall 0.61). **Image 116** is similar (recall 0.66). The ground truth is uncertain in both tray images.
- **Image 135** (bubbling water): the egg is only a faint shape under the bubbles, and the output covers only its upper left part (recall 0.47). Both outlines are irregular here.

4. What went wrong in the algorithm

- **Accept-and-union design.** Extra accepted blobs are never removed (62, 153, 129, 145). The same design caused the false blobs in trial T10 of Task 2.
- **Pale-egg assumption.** The brightness gate removes dark eggs (24).
- **Brightness and shape are the only checks.** A strongly coloured, egg-shaped picture passes (22) and white cables or foam next to a white egg get merged (62, 153).
- **Shading.** A candidate stops where the shading of the egg starts (126, 107), and GrabCut cannot grow it because it only refines a seed that already exists.
- **Ground truth.** In the egg-tray images (42, 116, 117) the Task 1 masks are ambiguous, so part of the measured error may be in the ground truth.

5. Suggested improvements (not applied)

The eval split has been used once, so I did not change the algorithm after seeing these results. Each idea below is untested. It should first be tried on dev, and a new version would need new held-out images or a new split for a clean final score.

- **Shape check after GrabCut.** Drop components with low solidity and remove thin parts with a morphological opening. The Task 2 diagnostic showed that eggs are almost perfectly convex (median solidity 1.0). This targets images 62, 153 and 145. It would not remove a convex patch such as the one in image 129.
- **Colour check.** Reject candidates with high colour saturation, since the eggs here are close to white. This targets image 22.
- **Dark-egg mode.** If no pale candidate fits well, look for dark candidates with a good ellipse fit and a plausible size, and exclude skin-coloured regions. This targets image 24.
- **Complete the shaded side.** Grow or complete the GrabCut result with its convex hull or a fitted ellipse after the refinement. In Task 2 the ellipse fill before GrabCut (T8) did not help, so this needs a test. It targets images 126 and 107.
- **Better ground truth** for the egg-tray images, by checking them by hand.

6. Limitations of this analysis

- Only 22 eval images, so one image moves the mean IoU by about 0.045.
- The qualitative findings come from the overlays of 12 images; the others are covered by numbers only.
- The dev split is compared with IoU only, because the dev predictions were not saved.
- The Task 1 masks are model predictions and not human labels.
- The eval split has no touching eggs, so one of the known weaknesses of the union mask is not measured.

7. AI tool disclosure

I used Claude (Anthropic) as an analysis and coding assistant for Task 3: to propose the metrics, to draft the metric and plotting code, and to help read the plots and overlays. I ran every cell myself in Colab, looked at the outputs, and wrote down the findings and hypotheses above.

Task 4: Tracking Individual Eggs in a Video

1. Objective

Task 4 asks me to run my final algorithm on a YouTube video (Mocomi, Raw and Boiled Egg Spinning, https://youtu.be/Y4JyV94y2_A) and track the individual eggs separately, modifying the segmentation and/or adding a tracking algorithm as needed. This section documents the video, the detector and tracker I used, every attempt (including the ones that failed), the final result and its limits.

Output video (public Google Drive link):
[https://drive.google.com/file/d/1xBU2eQbkeQ98u3YnsAMKMuvQ8GlEF2BZ/view?usp=sharing]

2. The video

1920×1080, 24 fps, 1,442 frames (60.1 s). I looked at one frame per second to see what it contains:

- 0–9 s: intro screen, no egg.
- About 10–42 s: two white eggs on a dark brown table, static camera. A hand moves one egg to the other side, lifts the other egg, and finally makes one egg spin.
- 42 s fade, 43–56 s a text slide, 57–59 s end card: no egg.

I processed frames 240 to 1000 (10 s to 41.7 s), every second frame, resized to 640×360. The output plays at 12 fps, which keeps real-time speed. The frames without eggs are skipped on purpose.

The two eggs look the same (white, similar size), so identity cannot come from appearance. It has to come from position and continuity, and I assume exactly two eggs at the start.

3. Approach

3.1 Detector

Each frame goes through my frozen Task 2 pipeline (threshold sweep with ellipse scoring, brightness gate, GrabCut refinement), wrapped in a function seg_video. It keeps components of at least 800 pixels whose centre is below y = 150 (this removes the shirt in the upper part of the frame).

3.2 Tracker (Tracker4)

- At the start frame the two masks get IDs from left to right: ID 1 is the left egg and ID 2 the right egg. No new tracks are created later.
- Each frame, every (track, mask) pair gets a cost = distance between centroids minus 300 × mask IoU. A pair is not allowed if IoU is below 0.2 and the distance is over 260 px, unless the track is already missing (so a hidden egg can be matched again after it was covered).
- Pairs are assigned greedily from the lowest cost. A track with no mask is marked as missed and keeps its last position.

4. Experiments

Step	Change	Why	Result	Decision
Tracker2	Hungarian matching on the cost above	First version	No result recorded in my notes	Replaced by Tracker3
Tracker3	Greedy matching, tracks with fewer missed frames first	Alternative assignment order	Trace of frames 480–558: when only one mask was found at frame 504, the first track in the order took it although the other track fitted it better (ID swap)	Discarded
Tracker4	Greedy matching on the globally lowest cost first	Fix the order problem of Tracker3	The same trace stays correct: the static egg keeps ID 2 through frames 504–552, and ID 1 returns at frame 558 (x about 403)	Kept

Step	Change	Why	Result	Decision
First full run	seg_video + Tracker4, frames 240–1000	Check the whole video	IDs 1 and 2 only, but 16 frames without a mask (30.3–32.3 s), and a visual check of frames 720–756 showed an ID swap at about 31 s	Problem found
Diagnosis	Looked at the track log and at 12 frames (30–31 s)	Find the cause	The left egg was hidden under the fingers and not detected (27–30 s). At frame 750 the egg at x = 70 was nearest to ID 2 (about 70 px) and far from ID 1 (about 360 px), so the tracker gave it ID 2. The two IDs were swapped from then on	
V1	Candidate minimum area from 2% to 1% of the frame	The visible part of the left egg is about 2,500–2,900 px (1.1–1.3%), below the 2% limit of 4,608 px; the static egg is 4,695 px (2.04%)	Failed. The Mocomi logo (bottom right, about 582, 339) became a candidate and took ID 1 from frame about 650 on. The left egg was still missing at 702, 714 and 720	Discarded alone

Step	Change	Why	Result	Decision
Diagnostic	Listed the accepted blobs before GrabCut and raised the cap from 3 to 6 blobs	Find where the left egg is lost	At 702 and 720 the cap was used up by the shirt, the static egg and the logo; with cap 6 the left egg appears (about 86, 190; 2,500–2,900 px). At 714, 726 and 738 it still does not, because it merges with the fingers into one large blob	
V2	Three changes together: minimum area 1%, cap 6, and an exclusion zone for blobs centred at x of at least 520 and y of at least 300 (logo)	Detect the partly hidden left egg without the logo	Frames 600–830: never more than 2 masks, only IDs 1 and 2, left egg found at 702 and 720 and kept as ID 1, spinning egg gets ID 2 at 798–828. The criterion I fixed in advance (every frame 810–830 with ID 1 on the left and ID 2 on the right) was met in 10 of 11 frames; in frame 826 the left egg was not detected	Kept

V2 bundles three changes, which breaks my one-change-per-trial rule, and I did not test them separately. The criterion was written before I saw the V2 output.

5. Final result

V2 with Tracker4 on frames 240–1000 (380 frames processed):

- Only IDs 1 and 2 appear, and no frame has more than 2 masks, so the logo, the shirt, the reflections and the text overlays are not tracked.
- 5 frames have no mask at all. ID 1 is present in 310 frames and ID 2 in 347.
- The per-second positions give a consistent story. ID 1 is the egg that starts on the left. The hand lifts it at 21 s, it is on the right at 24–27 s, it is hidden at 28–29 s, it is back on the left at 30 s, and it stays there (x about 71) until 41 s. ID 2 is the egg that stays at x about 157 until the hand lifts it at about 31 s; it is hidden at 31–33 s and then spins on the right (x about 390–430) until 41 s.
- I checked 12 frames of the annotated video by eye (12 s to 41 s, around the hand contacts). The label is on the correct egg in all of them, and none is on the shirt, the logo, the text or a reflection.

6. Limitations

- **Identity under occlusion is inferred, not observed.** The IDs stay consistent because there are only two eggs and at least one is visible at any time, so elimination fixes the identity. This fails if both eggs are hidden and moved, or if a third object appears.
- **No quantitative score.** There is no ground truth for the video; I inspected 12 of the 380 processed frames and the track log, but computed no tracking metric.
- **Gaps.** The covered egg is missing in some frames while the hand is on it (ID 1 at 22–23 s and 28–29 s, ID 2 at 31–33 s), and a few single frames (for example 826).
- **Mask quality.** The outline of the spinning egg is jagged from 34 s on. The video frames show stripes on the moving egg, so this is probably blur or compression in the source, but I did not check. Masks next to the hand can be partial.
- **Video-specific settings.** The 1% area limit, the cap of 6, the logo zone, the y filter at 150, the resize to 640×360 and the assumption of two eggs at the start were set by hand for this video. The segmentation itself is the frozen Task 2 pipeline, not retuned.
- **Coverage.** Only frames 240–1000 (every second frame) are processed; the intro, the text slides and the end card have no egg.

7. Reproducibility

Files: video_lib.py (seg_video_v3, Tracker4, run_video2), track_log_v2.csv (frame, ID, x, y) and tracked_v2.mp4. Colab was blocked by YouTube, so I downloaded the video on my laptop with yt-dlp (it needed Deno and the option --remote-components ejs:github), uploaded it to Google Drive and copied it into Colab.

8. AI tool disclosure

I used Claude (Anthropic) as a coding and analysis assistant for Task 4: to suggest the tracker and the diagnostics and to draft the code. I ran every cell myself, looked at the frames and the logs, and decided what to keep or discard.